



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

EXPERIMENT-1

Aim: 1a) Construct an AVL tree for a given set of elements which are stored in a file and implement insert and delete operation on the constructed tree

Program:

```
#include<stdio.h>
#include<stdlib.h>
//int maxi(int,int);
struct TreeNode
{
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
    int height;
};
int height(struct TreeNode *node)
{
    if (node==NULL)
        return 0;
    else
        return (node->height);
}

int maxi(int a,int b)
{
    return ((a>b)?a:b);
}
struct TreeNode* createNode(int key)
{
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    if (newNode != NULL)
    {
        newNode->data = key;
        newNode->left = NULL;
        newNode->right = NULL;
        newNode->height = 1; // New node is initially at height 1
    }
}
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
}  
return newNode;  
}  
  
struct TreeNode* rightRotate(struct TreeNode* y)  
{  
    struct TreeNode* x = y->left;  
    struct TreeNode* T2 = x->right;  
    x->right = y;  
    y->left = T2;  
    y->height = maxi(height(y->left), height(y->right)) + 1;  
    x->height = maxi(height(x->left), height(x->right)) + 1;  
    return x;  
}  
struct TreeNode* leftRotate(struct TreeNode* x)  
{  
    struct TreeNode* y = x->right;  
    struct TreeNode* T2 = y->left;  
  
    y->left = x;  
    x->right = T2;  
    x->height = maxi(height(x->left), height(x->right)) + 1;  
    y->height = maxi(height(y->left), height(y->right)) + 1;  
    return y;  
}  
  
int getBalance(struct TreeNode* node)  
{  
    if (node == NULL) return 0;  
    return height(node->left) - height(node->right);  
}  
  
struct TreeNode* insert(struct TreeNode* root, int key)  
{  
    int balance;  
    if (root == NULL) return createNode(key);  
    if (key < root->data) root->left = insert(root->left, key);  
    else if (key > root->data) root->right = insert(root->right, key);  
    else // Duplicate keys not allowed  
        return root;
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
// Update height of the current node
root->height = 1 + maxi(height(root->left), height(root->right));
balance = getBalance(root);
// Left Left Case

if (balance > 1 && key < root->left->data) return rightRotate(root);
// Right Right Case
if (balance < -1 && key > root->right->data) return leftRotate(root);
// Left Right Case
if (balance > 1 && key > root->left->data)
{
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
// Right Left Case
if (balance < -1 && key < root->right->data)
{
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
return root;
}

// Function to find the node with the minimum value
struct TreeNode* minValueNode(struct TreeNode* node)
{
    struct TreeNode* current = node;
    while (current->left != NULL) current = current->left;
    return current;
}

struct TreeNode* deleteNode(struct TreeNode* root, int key)
{
    {
        int balance;
        if (root == NULL) return root;
        // Perform standard BST delete
        if (key < root->data) root->left = deleteNode(root->left, key);
        else if (key > root->data) root->right = deleteNode(root->right, key);
        else {
            // Node with only one child or no child
            if ((root->left == NULL) || (root->right == NULL))
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
{
struct TreeNode* temp = root->left ? root->left : root->right;
// No child case
if (temp == NULL)
{
temp = root;
root = NULL;
} else // One child case
*root = *temp; // Copy the contents of the non-empty child
free(temp);
}
else
{
// Node with two children, get the inorder successor
struct TreeNode* temp = minValueNode(root->right);
// Copy the inorder successor's data to this node
root->data = temp->data;
// Delete the inorder successor
root->right = deleteNode(root->right, temp->data);
}
}

// If the tree had only one node, then return
if (root == NULL) return root;
// Update height of the current node
root->height = 1 + maxi(height(root->left), height(root->right));
// Get the balance factor to check whether this node became unbalanced
balance = getBalance(root);
// Left Left Case
if (balance > 1 && getBalance(root->left) >= 0) return rightRotate(root);
// Left Right Case
if (balance > 1 && getBalance(root->left) < 0)
{
root->left = leftRotate(root->left);
return rightRotate(root);
}
// Right Right Case

if (balance < -1 && getBalance(root->right) <= 0)
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
return leftRotate(root);

// Right Left Case
if (balance < -1 && getBalance(root->right) > 0)
{
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
return root;
}

// Function to perform in-order traversal of the AVL tree
void inOrderTraversal(struct TreeNode* root)
{
    if (root != NULL)
    {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}

// Function to free the memory allocated for the AVL tree
void freeAVLTree(struct TreeNode* root)
{
    if (root != NULL)
    {
        freeAVLTree(root->left);
        freeAVLTree(root->right);
        free(root);
    }
}

int main()
{
    struct TreeNode* root = NULL;
    int choice, key;
    do
    {
        printf("\nAVL Tree Operations:\n");
```



G.PULLA REDDY ENGINEERING COLLEGE (AUTONOMOUS): KURNOOL
DEPARTMENT OF EMERGING TECHNOLOGIES IN COMPUTER SCIENCE
B.TECH. – III SEMISTER (Scheme- 23)
LAB NAME(ADSA(P)) LABORATORY

```
printf("1. Insert a node\n");
printf("2. Delete a node\n");
printf("3. In-order Traversal\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
case 1:
printf("Enter the key to insert: ");

scanf("%d", &key);
root = insert(root, key);
break;
case 2:
printf("Enter the key to delete: ");
scanf("%d", &key);
root = deleteNode(root, key);
break;
case 3:
printf("In-order Traversal: ");
inOrderTraversal(root);
printf("\n");
break;
case 4:
// Free allocated memory
freeAVLTree(root);
printf("Exiting...\n");
break;
default:
printf("Invalid choice! Please enter a valid option.\n");
}
} while (choice != 4);
return 0;
}
```